Uppsala University



Machine Learning

Assignment 8

General information

- The recommended tool in this course is R (with the IDE R-Studio). You can download R **here** and R-Studio **here**. You can use Python and Jupyter Notebooks, although the assignments may use data available only through the R package, a problem you would need to solve yourself.
- Report all results in a single, `*.pdf`-file. *Other formats, such as Word, Rmd, Jupyter Notebook, or similar, will automatically be failed.* Although, you are allowed first to knit your document to Word or HTML and then print the assignment as a PDF from Word or HTML if you find it difficult to get TeX to work.
- The report should be written in English.
- If a question is unclear in the assignment. Write down how you interpret the question and formulate your answer.
- You should submit the report to **Studium**. Deadlines for all assignments are **Sunday 23.59**. See **Studium** for dates. Assignments will be graded within 10 working days from the assignment deadline.
- To pass the assignments, *you should answer all questions not marked with **, and get at least 75% correct.
- To get VG on the assignment, *also the questions marked with a ** should be answered. Sometimes you can choose between different VG assignments, then this is explicitly stated. To get VG you need 75% both on questions to pass and the VG part of the assignment. VG will only be awarded on the first deadline of the assignment.
- A report that does not contain the general information (see the **template**), will be automatically rejected.
- Every assignment report must include one oral exam question of your own together with a short model answer, following the oral exam question contribution instructions in this assignment. You are not allowed to use large language models, such as ChatGPT, at all for this contribution.
- State clearly whether the course staff may use or adapt your question and answer in a future oral exam: **Yes** or **No**.
- When working with R, we recommend writing the reports using R markdown and the provided **R markdown template**. The template includes the formatting instructions and how to include code and figures.
- Instead of R markdown, you can use other software to make the pdf report, but you should use the same instructions for formatting. These instructions are also available in **the PDF produced from the R markdown template**.
- The course has its own R package `uuml` with data and functionality to simplify coding. To install the package just run the following:

```
1. install.packages("remotes")
```

2. `remotes::install_github("MansMeg/IntroML", subdir = "rpackage")`

- We collect common questions regarding installation and technical problems in a course Frequently Asked Questions (FAQ). This can be found **here**.
- You are not allowed to show your assignments (text or code) to anyone. Only discuss the assignments with your fellow students. The student that show their assignment to anyone else could also be considered to cheat. During scheduled support sessions held on Zoom, only share your screen in a separate room with a teaching assistant.
- You may use large language models (LLMs), such as ChatGPT, like you may discuss with fellow students: to learn, ask for explanations, and get feedback. You may not use LLMs to produce the solution you submit. You must be able to explain every part of your own assignment at the oral exam, including your code, results, and reasoning, even when it is not covered by the published oral exam questions. If course staff have concerns about your understanding or the authorship of any part of your submission, you may be asked to explain that part at the oral exam.
- Where an exercise explicitly asks you to run an LLM and report its output, you may do so for that experiment. Your explanations and analysis must be your own. This permission does not apply to the oral exam question contribution, for which LLM use is prohibited entirely.
- Do not copy or upload the full assignment text into an LLM.
- Scheduled support sessions are for questions about the assignments and course material. Do not hesitate to ask! Ask questions early if instructions are unclear or you encounter possible bugs, so you do not lose time unnecessarily.
- If you have any suggestions or improvements to the course material, please post in the course chat feedback channel, create an issue **here**, or submit a pull request to the public repository.

Contents

Automatic feedback on your R functions 3

Oral exam question contribution 4

1 Bandits 5

2 Markov Decision Processes 11

3 * Some additional bandit algorithms 13

Automatic feedback on your R functions

The R package `markmyassignment` (MMA) provides automatic feedback on the functions listed in the selected check configuration. Define or load your own functions in R before running the checks. Use the `tasks` argument to check one function by name; omitting it checks every function in the selected configuration. Functions that depend on other functions need those definitions loaded too.

To select the checks for this assignment, run the following code in R:

```
install.packages("markmyassignment") # Install once
library(markmyassignment)
assignment_path <-
  paste("https://github.com/MansMeg/IntroML/",
        "blob/master/assignments/tests/assignment8/assignment8.yml", sep="")
set_assignment(assignment_path)
# List the function names available in this configuration
show_tasks()
# To check one function, replace function_name with its name:
# mark_my_assignment(tasks = "function_name")
# To check your code/functions, just run
mark_my_assignment()
```

The checks provide feedback on selected numerical cases; they do not assess the whole assignment or award marks. Print your complete function definitions in the PDF report as required by the general instructions, even when the checks pass. You do not need to include MMA results in the report. The supplied checks run R code; Python implementations are assessed from the report and cannot be run directly through MMA.

Run your function definitions first; use the task-specific checks below as you implement them. The checks do not assess your simulation experiments.

The algorithm checks use fixed rewards, deterministic transitions, or properties that hold for any sampled action. The reward-generator checks use sample means and standard deviations with tolerances for sampling variation. They do not require the same random choices as the examples. Different correct implementations can consume random numbers or break ties differently, so their outputs can differ even with the same seed.

Oral exam question contribution

Write one possible oral exam question of your own for this assignment together with a short model answer. Include both in your assignment report.

The question should:

- concern material from this assignment or the associated reading or lectures,
- be suitable for a 2–4 minute oral answer, possibly using a whiteboard,
- test conceptual understanding rather than only calculation or code,
- not already be included in the oral exam question list and not be very close to an existing question (e.g. just different numbers).

Note! You are not allowed to use large language models, such as ChatGPT, at all for this oral exam question contribution.

State clearly whether the course staff may use or adapt your question and answer in a future oral exam: **Yes** or **No**.

1 Bandits

In the first part, we will study the k-armed bandit problems more in detail.

Example: Dynamic web page example application *No submission required*

A website chooses which of five headlines to show. Each headline is an action; a click gives reward 1 and no click gives reward 0. The website observes the response only to the headline shown, so it must balance using promising headlines with trying others. Changing reader interests motivate the non-stationary setting.

1. (2p) **Implementation (1p).** Implement a stationary 5-armed bandit reward function that generates a reward R for a given category according to a true underlying (stationary) reward. Implement a function `stationary_bandit(a, t = NULL)` that takes an action $\mathcal{A} = (1, ..., 5)$ and ignores the optional time argument `t`. The reward function should return a reward according to

$$R \sim N(q_a^\star, 1)\,,$$

where $q^\star = (1.62, 1.20, 0.70, 0.72, 2.03)$. Below is an example of how it *could* work. Note that since the function returns a random value, you might get a different result, and it might still be a correct implementation. However, to test, you can run your bandit 1000 times and take the mean (see below). Then you should be close to $q^\star$ if your implementation is correct. **Example (1p).** Include inputs, a call, seed and compact actual output from your implementation, with a brief explanation. Reuse a supplied example or the required experiments.

```
set.seed(4711)
stationary_bandit(2)
## [1] 3.019735
stationary_bandit(1)
## [1] 2.99044
R <- numeric(1000)
for (i in 1:1000) {
    R[i] <- stationary_bandit(3)
}
mean(R)
## [1] 0.6819202
```

Check: `mark_my_assignment(tasks = "stationary_bandit")`

2. (2p) **Implementation (1p).** Implement a non-stationary reward function. For all actions $\mathcal{A} = (2, ..., 5)$ the non-stationary should work as the stationary reward function. Although for action $a = 1$, the function should return a reward based on the iteration according to

$$R \sim N(-3 + 0.01t, 1)\,.$$

Implement the reward function as `nonstationary_bandit(a, t)` and it should work as follows. Again, note that these are random values, you might have another (correct) solution giving different results. **Example (1p).** Include inputs, a call, seed and compact actual output from your implementation, with a brief explanation. Reuse a supplied example or the required experiments.

```
set.seed(4711)
nonstationary_bandit(2, t = 1)
## [1] 3.019735
nonstationary_bandit(2, t = 1000)
## [1] 2.57044
nonstationary_bandit(1, t = 1)
## [1] -1.793682
nonstationary_bandit(1, t = 1000)
## [1] 6.593121
```

Check: `mark_my_assignment(tasks = "nonstationary_bandit")`

3. (2p) **Implementation (1p).** Implement a greedy policy algorithm that always chooses the greedy action, according to Eq. (2.2) in Sutton and Barto (2020). Use sample-average value estimates. The function called `greedy_algorithm(Q1, bandit)` should input a vector of initial estimates Q_1 and a reward function. Then the algorithm should run the bandit for 1000 steps and return a list containing (1) The rewards R_t for all steps, (2) the mean reward over all the 1000 steps, (3) the value estimates $Q_t(a)$, and (4) the total number of choices of each action $N_t(a)$. Here is an example of how it could work. *Note!* This is a random algorithm so you might get a different result, even if you use the exact same seed and have a correct algorithm. All bandit algorithms below should request each reward with `bandit(a, t = i)`, where `i` is the current step (1 to 1000); both reward functions accept this call. **Example (1p).** Include inputs, a call, seed and compact actual output from your implementation, with a brief explanation. Reuse a supplied example or the required experiments.

```
Q1 <- rep(0, 5)
set.seed(4711)
greedy_algorithm(Q1, stationary_bandit)
## $Qt
## [1] 1.602744 0.000000 0.000000 0.000000 0.000000
##
## $Nt
## [1] 1000    0    0    0    0
##
## $R_bar
## [1] 1.602744
##
## $Rt
## [1] 3.439735 2.990440 2.816318 1.213121 ...

set.seed(4711)
Q1 <- rep(0, 5)
greedy_algorithm(Q1, nonstationary_bandit)
## $Qt
## [1] -1.170265  1.180905  0.000000  0.000000  0.000000
##
## $Nt
## [1]   1 999   0   0   0
##
## $R_bar
## [1] 1.178554
##
## $Rt
## [1] -1.1702649  2.5704395  2.3963182  0.7931208 ...
```

Check: `mark_my_assignment(tasks = "greedy_algorithm")`

Hint: Checking returned results *No submission required*

Check the number of rewards, total action count, and reported mean:

```
result <- greedy_algorithm(rep(0, 5), stationary_bandit)

stopifnot(
  length(result$Rt) == 1000,
  sum(result$Nt) == 1000,
  isTRUE(all.equal(result$R_bar, mean(result$Rt)))
)
```

No output means these checks passed. Reuse them for the later bandit algorithms.

4. (2p) **Implementation (1p).** Implement the ϵ-greedy algorithm.

   Use `epsilon_algorithm(Q1, bandit, epsilon)`. With probability ϵ, choose an action uniformly from all five actions; otherwise choose a greedy action. Keep the greedy algorithm's updates and output structure. **Example (1p).** Include inputs, a call, seed and compact actual output from your implementation, with a brief explanation. Reuse a supplied example or the required experiments.

```
set.seed(4711)
Q1 <- rep(0, 5)
epsilon_algorithm(Q1, stationary_bandit, 0.1)
## $Qt
## [1] 1.6093183 1.1491153 0.7382012 0.5975228 2.0272784
##
## $Nt
## [1] 234  17  19  16 714
##
## $R_bar
## [1] 1.867178
##
## $Rt
## [1] 1.7725176 2.8163182 0.6335086 2.0175494 ...

set.seed(4711)
Q1 <- rep(0, 5)
epsilon_algorithm(Q1, nonstationary_bandit, 0.1)
## $Qt
## [1] 1.5887316 1.0956679 0.6962471 0.5975228 2.0468788
##
## $Nt
## [1]  24 159  20  16 781
##
## $R_bar
## [1] 1.834438
##
## $Rt
## [1] -2.8374824  2.3963182  0.2135086  2.0175494 ...
```

Check: `mark_my_assignment(tasks = "epsilon_algorithm")`

5. (2p) **Implementation (1p).** Implement constant-step ϵ-greedy updates, as in Eq. (2.5) in Sutton and Barto (2020), as `nonstationary_algorithm(Q1, bandit, epsilon, alpha)`. Return the same output structure as the greedy algorithm. **Example (1p).** Include inputs, a call, seed and compact actual output from your implementation, with a brief explanation. Reuse a supplied example or the required experiments.

```
set.seed(4711)
Q1 <- rep(0, 5)
nonstationary_algorithm(Q1, stationary_bandit, epsilon = 0.1,
    alpha = 0.2)
## $Qt
## [1] 1.414101 1.293336 1.237085 0.517214 2.134716
##
## $Nt
## [1] 302  16  19  16 647
##
## $R_bar
## [1] 1.840128
##
## $Rt
## [1] 1.7725176 2.8163182 0.6335086 2.0175494 ...

set.seed(4711)
Q1 <- rep(0, 5)
nonstationary_algorithm(Q1, nonstationary_bandit, epsilon = 0.1,
    alpha = 0.2)
## $Qt
## [1] 7.055805 1.287957 1.243996 0.517214 2.569759
##
## $Nt
## [1] 387 193  70  16 334
##
## $R_bar
## [1] 2.818588
##
## $Rt
## [1] -2.8374824  2.3963182  0.2135086  2.0175494 ...
```

Check: `mark_my_assignment(tasks = "nonstationary_algorithm")`

6. (2p) Evaluate each setting below with both the stationary and non-stationary bandit. Use $Q_1(a) = 0$ throughout. Run 500 simulations per combination, with 1000 steps per simulation. Report the average of the 500 simulation mean rewards in a table with four setting rows and two environment columns: eight results in total.

Setting	Parameters
Greedy	—
ϵ-greedy	$\epsilon = 0.1$
Constant-step ϵ-greedy	$\epsilon = 0.1,\ \alpha = 0.5$
Constant-step ϵ-greedy	$\epsilon = 0.1,\ \alpha = 0.9$

Hint: Running simulations efficiently *No submission required*

While debugging, start with five simulations. Preallocate the reward matrix and fill one column per simulation. Once your checks pass, use 500 simulations for every combination in your reported results.

7. (2p) What are your conclusions from these results? Which setting performs best in each environment, and why?

8. (2p) For each environment, choose the best and worst settings using your results from Exercise 1.6. Plot their average reward per step, as in Figure 2.2 in Sutton and Barto (2020). At each time step, average the rewards across the 500 simulations and plot these 1000 averages. Reuse the rewards R_t already simulated.

```
# rewards: 1000 rows (steps), 500 columns (simulations)
average_reward_per_step <- rowMeans(rewards)
```

Hint: Interpreting learning curves *No submission required*

Early behaviour reflects initial estimates and exploration: trying actions may reduce immediate reward but reveal better choices. Later, consider exploitation and how quickly estimates adapt to changing rewards. In the non-stationary setting, a rising curve can also reflect action 1's increasing expected reward.

2 Markov Decision Processes

In a Markov Decision Process (MDP), the agent chooses an action A_t based on the current state S_t. The environment then produces the next state S_{t+1} and reward R_{t+1}. We use the recycling robot MDP (Example 3.7 in Sutton and Barto, 2020).

We write p_{high} for the probability of remaining high after searching from high energy, and p_{low} for the probability of remaining low after searching from low energy. These correspond to the transition probabilities called α and β in the textbook.

1. Use `recycling_mdp(p_high, p_low, r_wait, r_search)` in the `uuml` R package to access the MDP in Sutton and Barto (2020). Use version 0.5.0 or later; reinstall using the instructions at the start of the assignment if needed.

```
library(uuml)
mdp <- recycling_mdp(p_high = 0.5, p_low = 0.8, r_wait = 0.1,
    r_search = 1)
mdp
##        s s_next         a p_s_next r_s_a_s_next
## 1   high   high    search      0.5          1.0
## 2   high    low    search      0.5          1.0
## 3    low   high    search      0.2         -3.0
## 4    low    low    search      0.8          1.0
## 5   high   high      wait      1.0          0.1
## 6   high    low      wait      0.0          0.1
## 7    low   high      wait      0.0          0.1
## 8    low    low      wait      1.0          0.1
## 9    low   high recharge      1.0          0.0
## 10   low    low recharge      0.0          0.0
```

Example: Recycling robot transitions *No submission required*

With $p_{\text{high}} = 0.5$ and $p_{\text{low}} = 0.8$ in the call above:
Search with high energy: 50% probability of remaining high and 50% of becoming low. Either transition gives reward 1.
Search with low energy: 80% probability of remaining low with reward 1; 20% of requiring rescue, returning to high energy with reward -3.

2. (2p) **Implementation (1p).** Implement `always_search_policy(mdp)` to simulate 1000 actions, starting in state `high` and always choosing `search`. Return a list with mean reward `R_bar` and state counts `Nt`, named `high` and `low`. Count the state before each action. **Example (1p).** Include inputs, a call, seed and compact actual output from your implementation, with a brief explanation. Reuse a supplied example or the required experiments.

```
set.seed(4711)
always_search_policy(mdp)
## $Nt
## high  low
##  288  712
##
## $R_bar
## [1] 0.452
```

Check: `mark_my_assignment(tasks = "always_search_policy")`

3. (2p) **Implementation (1p).** Implement `charge_when_low_policy(mdp)` using the same initial state, run length, and output structure. Choose `recharge` when energy is low and `search` when it is high.

   Example (1p). Include inputs, a call, seed and compact actual output from your implementation, with a brief explanation. Reuse a supplied example or the required experiments. Reuse a policy run from the comparison below.

   Check: `mark_my_assignment(tasks = "charge_when_low_policy")`

4. (2p) Compare the two policies for MDP with $p_{\text{high}} = p_{\text{low}} = 0.9$ and $p_{\text{high}} = p_{\text{low}} = 0.4$ with $r_{\text{wait}} = 0.1$ and $r_{\text{search}} = 1$. Run at least ten times per policy in each MDP and compute the mean reward. Which policy performs better in each MDP?

3 * Some additional bandit algorithms

This task is only necessary to get one point toward a *pass with distinction* (VG) grade. Hence, if you do not want to get a *pass with distinction* (VG) point, you can ignore this part of the assignment.

We are here going to implement two additional bandits and compare them to the previous implementations above.

Select the VG checks before checking these functions:

```
set_assignment(paste("https://github.com/MansMeg/IntroML/",
                     "blob/master/assignments/tests/assignment8/",
                     "assignment8-vg.yml", sep = ""))
```

Reading recommendations: Read Section 2.8, "Gradient Bandit Algorithms", in Sutton and Barto (2020).

1. (2p) **Implementation (1p).** Implement `ucb_algorithm(Q1, bandit, c)` using UCB with sample-average value estimates. Use the greedy algorithm's run length and output structure. Choose an untried action whenever any $N_t(a) = 0$; otherwise maximize the UCB score. Any tie order is valid. The example chooses the first untried or maximizing action. **Example (1p).** Include inputs, a call, seed and compact actual output from your implementation, with a brief explanation. Reuse a supplied example or the required experiments.

```
set.seed(4711)
Q1 <- rep(0, 5)
ucb_algorithm(Q1, stationary_bandit, c = 2)
## $Qt
## [1] 1.5181372 1.0663840 0.6525280 0.1554593 2.0256255
##
## $Nt
## [1]  57  22  13   7 901
##
## $R_bar
## [1] 1.944654
##
## $Rt
## [1] 3.4397351 2.5704395 1.8963182 0.3131208 ...

set.seed(4712)
Q1 <- rep(0, 5)
ucb_algorithm(Q1, stationary_bandit, c = 2)
## $Qt
## [1] 1.15277323 1.18612642 0.48569375 0.05933926 1.98826399
##
## $Nt
## [1]  28  31  10   8 923
##
## $R_bar
## [1] 1.909547
##
## $Rt
## [1]  1.98183569  1.12125661  0.92862700 -0.01161757 ...
```

Check: `mark_my_assignment(tasks = "ucb_algorithm")`

2. (2p) **Implementation (1p).** Implement the gradient bandit algorithm in Sutton and Barto (2020, Section 2.8).

   Use `gradient_bandit_algorithm(bandit, alpha)`. Initialize all $H_1(a) = 0$. After observing R_t, update the baseline to include the current reward:

$$\bar{R}_t = \frac{1}{t} \sum_{s=1}^{t} R_s.$$

   Then update the preferences using $R_t - \bar{R}_t$ and the action probabilities used to select the current action. Recompute softmax probabilities afterwards. At the first step, $\bar{R}_1 = R_1$, so the preference update is automatically zero. Return the same output structure as the greedy algorithm, replacing `Qt` with the final preferences `Ht` and action probabilities `pit`. **Example (1p).** Include inputs, a call, seed and compact actual output from your implementation, with a brief explanation. Reuse a supplied example or the required experiments.

Example: Updating action preferences *No submission required*

After the first reward of 1, the baseline is 1 and preferences remain zero, so each action has probability 0.2. Suppose the second reward is 3 and $\alpha = 0.1$. Update the baseline to $(1 + 3)/2 = 2$, giving an advantage of $3 - 2 = 1$:

$$\begin{aligned} \text{Chosen action:} \quad & H(a) = 0 + 0.1(1)(1 - 0.2) = 0.08, \\ \text{Each other action:} \quad & H(a) = 0 - 0.1(1)(0.2) = -0.02. \end{aligned}$$

Applying softmax makes the chosen action more likely.

```r
set.seed(4711)
gradient_bandit_algorithm(stationary_bandit, alpha = 0.1)
## $Ht
## [1] -0.09767824 -1.00460839 -1.83627818 -1.90776481  4.84632962
##
## $pit
## [1] 0.007038550 0.002841898 0.001237140 0.001151788 0.987730625
##
## $Nt
## [1]  99  21  15  17 848
##
## $R_bar
## [1] 1.902805
##
## $Rt
## [1]  1.7725176  2.3963182 -0.2864914 -0.3089120 ...

set.seed(4712)
gradient_bandit_algorithm(stationary_bandit, alpha = 0.1)
## $Ht
## [1] -1.155096 -1.255883 -1.273164 -1.913755  5.597897
##
## $pit
## [1] 0.0011629534 0.0010514564 0.0010334422 0.0005446043 0.9962075437
##
## $Nt
## [1]  10  29  24  21 916
##
## $R_bar
## [1] 1.883679
##
## $Rt
## [1]  0.62963402  0.94862700 -0.30089716  0.08850766 ...
```

Check: `mark_my_assignment(tasks = "gradient_bandit_algorithm")`

3. (2p) Run the gradient bandit with $\alpha = 0$. Inspect the preferences `Ht`, action probabilities `pit`, and mean reward `R_bar`. Which quantities remain unchanged? Does the agent still receive rewards?

4. (2p) Compare UCB with $c = 0.5$ and $c = 2$ and the gradient bandit with $\alpha = 0.1$ and $\alpha = 1$ in both environments. Use $Q_1(a) = 0$ for UCB. Run 500 simulations of 1000 steps per combination and report the average of the simulation mean rewards: eight results in total. Compare with Exercise 1.6. Which setting performs best in each environment?

References

Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction (Second edition)*. MIT press, 2020.